claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_793a5d86-1c4\agent-a0c716ca0ccf5aead.jsonl`
- SHA-256: `d6b2df77a706c32d5de0ad7bcc4fa37e08acd93a770b2a020c6ad03a8e3d9b93`
- Source modified: `2026-07-02T08:18:53+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_793a5d86-1c4`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`

Transcript

1. user (2026-07-02T08:18:17.562Z)

You are an ADVERSARIAL verifier. Independently re-check the fate verdict for OPEN PR #415 ("api: root relative security.upload_dir under app home (R1-20, P0a parity)", branch origin/fix/upload-dir-app-home) on Khelsutra/badminton-highlight-indexer. Try to REFUTE the analyst.

Analyst's verdict: recommended_fate=CLOSE_SUPERSEDED; superseded_by=422 (confidence high).
Analyst's master_evidence: Master HEAD d00b4db. The PR's functional intent (route security.upload_dir through resolve_output_dir / app_home in _upload_cfg) is ALREADY present on master: backend/api/uploads.py:53-55 `upload_dir = resolve_output_dir(getattr(sec, "upload_dir", None) or "output/uploads")` with the enabling import at backend/api/uploads.py:24-26 `from backend.utils.app_home import resolve_output_dir`. Introduced by merged PR #422 (commit 4f148d1, "fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27)"), whose body explicitly states "R1-20: Fix upload_dir CWD-relative (never app-home-resolved) ... resolve upload_dir through resolve_output_dir() in _upload_cfg, mirroring output_dir." `git log -S resolve_output_dir -- backend/api/uploads.py` returns only 4f148d1. Resolver semantics validated in backend/utils/app_home.py:36-45 (RALLY_APP_HOME unset -> Path(".") CWD-verbatim; set -> relative roots under home) and resolve_output_dir at line 83. The ONLY parts of #415 still absent on master are non-functional: the app-home doc comment in backend/config/models.py:332-336 (master has the old comment, no RALLY_APP_HOME mention), one docs/CODE_MAP.md sentence, and 4 new tests in tests/test_upload_download.py (grep for test_upload_cfg_relative_default_roots_under_app_home / test_upload_lands_under_app_home_not_launch_cwd returns nothing on master).
Analyst's conflict_nature: CONFLICTING because master already did the same thing (superseded), not adjacent edits. PR branch merge-base is e410136 (parent line before #422). Both #415 and #422 edit the IDENTICAL lines: the resolve_output_dir import block and the `upload_dir = ...` assignment inside _upload_cfg in backend/api/uploads.py. #422's hunk (@@ line ~50) wraps the exact same expression in resolve_output_dir and adds the same import; #415 does the same with only cosmetic differences (single-line import + longer comment). Overlapping same-line edits on backend/api/uploads.py are the conflict. models.py/CODE_MAP.md/test hunks are additive and would not themselves conflict.
Analyst's rationale: The load-bearing fix (upload_dir routed through resolve_output_dir in _upload_cfg, plus its import) is already on master via merged PR #422 (4f148d1), which the branch predates. #415's uploads.py change is byte-equivalent in effect and conflicts precisely because master already applied it ? the classic two-sessions-same-fix collision. The only residual deltas are doc comments and extra tests describing/verifying behavior master already has; they do not justify a rebase of this whole PR. Close as superseded. If the owner values the extra app-home test coverage and doc-comment polish, cherry-pick just those additive hunks (tests/test_upload_download.py P0a section + models.py/CODE_MAP.md comment updates) as a tiny follow-up rather than rebasing #415.

Already MERGED onto master (verify against actual master code, do not assume):
- #422 = fix for --output-dir clobbering config.json output.output_dir (CODE_MAP updated for it)
- #423 = docs: health-remediation plan added + DOC_STATUS/README/NEXT_STEPS registered; audit marked snapshot

```
- #424 = P2: /api/process idempotent by content hash (#340 re-bill fix)
- #425 = P3: /api/source resolves remote source into output-scoped source_cache, reuses across
Range requests, warms annotation proxy
- #426 = P4: bounded ffprobe/ffmpeg subprocess timeouts on serving paths
- #427 = P5: clean remote-input scratch dirs after process/compile/assets/CLI; durable reports
- #428 = P1/P6: backend/storage/gcs.py lazy-import mypy typing fix
master HEAD = d00b4db "fix gcs storage lazy import typing (#428)"

Working dir: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer on master.
SHARED checkout ? READ-ONLY, no branch checkout, no working-tree changes.

Your job ? verify by looking at code yourself, do NOT take the analyst's word:
- If the verdict is CLOSE_SUPERSEDED: the risk is closing a PR whose work is NOT actually on
master (losing real work). Open the specific master file:line the analyst cited AND the PR diff
(gh pr diff 415 -R Khelsutra/badminton-highlight-indexer). Confirm master truly implements the
SAME intent (not just a similarly-named symbol). If the PR does anything master lacks (extra
tests, a stricter guard, a different file), flag it ? that means NOT fully superseded.
- If the verdict is TAKE_OVER_*: the risk is that the work IS already on master (would be
redundant/conflicting) OR the change is buggy. Confirm master still lacks it, and sanity-check
the change is correct.
- Default to skepticism: only agree=true if the evidence in code clearly supports it.

Return ONLY the structured object.
```

2. assistant (2026-07-02T08:18:20.222Z)

I'll independently verify the analyst's CLOSE_SUPERSEDED verdict by examining master code and the PR diff.

3. user (2026-07-02T08:18:24.317Z)

```
d00b4db fix gcs storage lazy import typing (#428)
---PR DIFF---
diff --git a/backend/api/uploads.py b/backend/api/uploads.py
index ccb1511..30be0b1 100644
--- a/backend/api/uploads.py
+++ b/backend/api/uploads.py
@@ -3,6 +3,9 @@
 Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
 proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
 under security.upload_dir, then submitted to /api/process by the returned path.
+A RELATIVE upload_dir roots under the app home (RALLY_APP_HOME) via the same
+``resolve_output_dir`` used for output.output_dir ? P0a semantics: env unset ?
+verbatim (CWD-relative, byte-identical to before); absolute ? passes through.
 Upload state is in-process by design (this is the single-box alpha): a server restart
 aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
 arrives with PR-3 identity scoping.
@@ -21,6 +24,7 @@
 from fastapi import APIRouter, HTTPException, Request
 from pydantic import BaseModel

+from backend.utils.app_home import resolve_output_dir  # P0a app-home rooting
 from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
 from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
 from backend.utils.paths import safe_output_filename
@@ -47,7 +51,14 @@ def _upload_cfg():
     import backend.main as _main

     sec = getattr(_main.global_config, "security", None)
-    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
+    # P0a: root a RELATIVE upload_dir under the app home (RALLY_APP_HOME), exactly like
+    # main() resolves output.output_dir ? the packaged `indexer --app` launches from an
+    # arbitrary shell CWD, and the upload landing zone must live with the DB/reels, not
+    # wherever the shell happened to be. Env unset ? verbatim (CWD-relative, unchanged);
+    # an absolute upload_dir always passes through untouched.
```

```
+    upload_dir = resolve_output_dir(
+        getattr(sec, "upload_dir", None) or "output/uploads"
+    )
     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
     exts = [
diff --git a/backend/config/models.py b/backend/config/models.py
index 652693a..e6b13e3 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -330,9 +330,12 @@ class SecurityConfig(BaseModel):
     allowed_video_root: Optional[str] = None

     # --- Chunked upload (B2, PR-2) ---
-    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
-    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
-    # contain upload_dir or /api/process will reject the uploaded paths.
+    # Landing zone for browser uploads (assembled from parts). A RELATIVE path roots
+    # under the app home (RALLY_APP_HOME) at use time ? same P0a resolve_output_dir
+    # semantics as output.output_dir (env unset ? CWD-relative verbatim; absolute ?
+    # passes through). Per-user subdirectories arrive with PR-3 identity scoping.
+    # ? In hosted mode, set allowed_video_root to contain upload_dir or /api/process
+    # will reject the uploaded paths.
     upload_dir: str = "output/uploads"
     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
     # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
diff --git a/docs/CODE_MAP.md b/docs/CODE_MAP.md
index 1aa5260..2657218 100644
--- a/docs/CODE_MAP.md
+++ b/docs/CODE_MAP.md
@@ -112,7 +112,7 @@ Entrypoints: `@run_eval.py` (single video score),
`@scripts/run_phase3_eval.py`
 - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI
mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
   - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals
set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every
endpoint NPEs.
   - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202
+ job_id**. `::_drain_due_jobs` / `::_run_claimed_job` (worker loop EXTRACTED to
`@backend/api/job_worker.py` (#234), re-exported on `backend.main`): claim each runnable job via
`db.claim_due_job` -> `_execute_processing` -> mark done/failed (a `JobWaiting` parks the job
via `set_job_waiting` for resume; EXECUTION_POLICY P3 self-scheduling). `::_execute_processing`
= the former sync body (hash -> validate -> add_video -> segment -> `finally:` always
`emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING
segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET
/api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job
`status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in
`result["status"]`. `::_get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)`
(single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs
`db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights`
`@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET
/api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the
pipeline synchronously inline (separate code path, unchanged by #16).
-  - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::_upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
```

part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete`
(`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB).
+ - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::_upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete`
(`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB). A RELATIVE `upload_dir` is rooted under the app home via
`resolve_output_dir` in `_upload_cfg` (P0a ? same semantics as `output.output_dir`:
`RALLY_APP_HOME` unset ? CWD-relative verbatim; absolute ? passthrough), so packaged `indexer
--app` uploads land with the DB/reels regardless of the launch CWD.
   - **STUDIO / STORAGE endpoints (P1/P2/P3/P4/P6/P8, 2026-07-01):** `::capabilities` `@GET
/api/capabilities` ? honest box probe; now includes a **`storage` block**
(`storage_capabilities` in `@backend/storage/capabilities.py`: local/s3/gcs/gdrive
`{id,kind,label,usable,free_gb}`, secret-free) alongside segmenters/ffmpeg/gpu/deployment.
`::list_videos` `@GET /api/videos` (the "my videos" list, P6). `::get_source` `@GET
/api/source/{id}` ? Range-capable FileResponse serving a **frame-identical annotation proxy**
(D4), source fallback, background-warmed. `::post_annotations` `@POST /api/annotations` ? writes
the verbatim annotator CSV to `output/labels/<video_id>.rallies.csv` (P8). `::create_asset`
`@POST /api/assets` (**P4**, `{uploaded_path?|source_uri?, medium_uri, sport?}`) ? stores a
video into a user-chosen medium (MD5-keyed `storage.put` for an upload?remote medium; register-
in-place for local; at-rest URI = no copy), registers it (video_id=MD5) so it shows in
`/api/videos`; **self-host posture only** (403 when `security.allowed_video_root` set); edge-
auth mirrors `/api/process`. Cost ledger: `::get_job_cost` `@GET /api/jobs/{id}/cost`,
`::get_video_cost` `@GET /api/videos/{id}/cost`. **Free-space guard (P2):**
`@backend/utils/freespace.py::require_free_bytes` -> HTTP **507** in `uploads.py::upload_init` +
the `/api/process` remote fetch-to-scratch; the WSL frame-extraction guard now **BLOCKS**
(`detectors/base.py::_wsl_frame_stage_shortfall`, ~15× source).
   - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly.
`::ProcessRequest`/`::QueryRequest`/`::CompileRequest`/`::AnnotationsRequest`/`::AssetRequest`
pydantic bodies.
   - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
diff --git a/tests/test_upload_download.py b/tests/test_upload_download.py
index 6be9145..9566e6b 100644
--- a/tests/test_upload_download.py
+++ b/tests/test_upload_download.py
@@ -9,6 +9,7 @@
 """

 import os
+from pathlib import Path

```
 from unittest.mock import patch

 import pytest
@@ -20,6 +21,7 @@
 from backend.api import uploads as uploads_api
 from backend.config.models import AppConfig
 from backend.infrastructure.database import Database
+from backend.utils import app_home as ah
 from backend.utils.hashing import compute_video_id


@@ -233,6 +235,65 @@ def test_abort_endpoint_removes_state(env):
     ]


+# --- P0a: upload_dir roots under the app home (RALLY_APP_HOME) --------------------------
+# Same resolve_output_dir semantics as output.output_dir: env unset ? verbatim
+# (CWD-relative, pre-P0a behaviour); env set ? relative roots under the home,
+# absolute passes through. Packaged `indexer --app` launches from any shell CWD.
+
+
+def test_upload_cfg_relative_default_roots_under_app_home(monkeypatch, tmp_path):
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
+    monkeypatch.setattr(m, "global_config", AppConfig())  # upload_dir at its default
+    upload_dir, _, _, _ = uploads_api._upload_cfg()
+    assert upload_dir == str(tmp_path / "output" / "uploads")
+
+
+def test_upload_cfg_absolute_upload_dir_passes_through(monkeypatch, tmp_path):
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
+    cfg = AppConfig()
+    abs_dir = str(tmp_path / "elsewhere")
+    cfg.security.upload_dir = abs_dir
+    monkeypatch.setattr(m, "global_config", cfg)
+    upload_dir, _, _, _ = uploads_api._upload_cfg()
+    assert upload_dir == abs_dir
+
+
+def test_upload_cfg_env_unset_keeps_cwd_relative_default(monkeypatch):
+    # RALLY_APP_HOME unset ? byte-identical to the pre-P0a CWD-relative default.
+    monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
+    monkeypatch.setattr(m, "global_config", AppConfig())
+    upload_dir, _, _, _ = uploads_api._upload_cfg()
+    assert upload_dir == "output/uploads"
+
+
+def test_upload_lands_under_app_home_not_launch_cwd(tmp_path, monkeypatch):
+    # End-to-end packaged shape: RALLY_APP_HOME set, upload_dir left at its relative
+    # default, server "launched" from an arbitrary shell CWD ? the assembled upload
+    # must land under <home>/output/uploads and leak nothing into the CWD.
+    home = tmp_path / "home"
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(home))
+    cwd = tmp_path / "some-shell-cwd"
+    cwd.mkdir()
+    monkeypatch.chdir(cwd)
+    db = Database(str(tmp_path / "t.db"))
+    cfg = AppConfig()  # security.upload_dir stays 'output/uploads'
+    cfg.output.output_dir = str(tmp_path / "out")
+    os.makedirs(cfg.output.output_dir, exist_ok=True)
+    monkeypatch.setattr(uploads_api, "_uploads", {})
+    client = TestClient(m.create_app(cfg, db))
+    up = client.post("/api/upload/init", json={"filename": "match.mp4"}).json()
+    r = client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"match-bytes")
+    assert r.status_code == 200
+    done = client.post(
```

```
+        f"/api/upload/{up['upload_id']}/complete", json={"total_parts": 1}
+    )
+    assert done.status_code == 200
+    final = Path(done.json()["path"])
+    assert final.parent.resolve() == (home / "output" / "uploads").resolve()
+    assert final.read_bytes() == b"match-bytes"
+    assert not (cwd / "output").exists()  # nothing written into the launch CWD
+
+
 # --- download (B3) ----------------------------------------------------------
```

4. user (2026-07-02T08:18:24.977Z)

```
4f148d1 fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27) (#422)
```

5. assistant (2026-07-02T08:18:28.154Z)

All PR hunks confirmed. Now let me verify master's actual state at the cited lines.

6. user (2026-07-02T08:18:31.722Z)

```
=== uploads.py 20-60 ===

from fastapi import APIRouter, HTTPException, Request
from pydantic import BaseModel

from backend.utils.app_home import (
    resolve_output_dir,  # P0a: app-home-aware path resolution
)
from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
from backend.utils.paths import safe_output_filename

router = APIRouter()

_uploads: Dict[str, Dict[str, Any]] = {}
_uploads_lock = threading.Lock()


class UploadInitRequest(BaseModel):
    filename: str
    total_size_bytes: Optional[int] = None


class UploadCompleteRequest(BaseModel):
    total_parts: int


def _upload_cfg():
    # Resolved lazily (import inside the function) to read the LIVE startup config that
    # main() sets on the module global, and to avoid a circular import at module load
    # (main.py imports this router; this module must not import main at import time).
    import backend.main as _main

    sec = getattr(_main.global_config, "security", None)
    upload_dir = resolve_output_dir(
        getattr(sec, "upload_dir", None) or "output/uploads"
    )  # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
    part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
    exts = [
        e.lower().lstrip(".")
        for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
=== app_home.py 30-90 ===
```

```
APP_HOME_ENV = "RALLY_APP_HOME"

#: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
APP_DIR_NAME = "Khelsutra"


def app_home() -> Path:
    """The resolved application-data home.

    ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD, byte-
identical
    to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
    """
    env = os.environ.get(APP_HOME_ENV)
    if env and env.strip():
        return Path(env.strip()).expanduser()
    return Path(".")


def app_path(*parts: str | os.PathLike[str]) -> Path:
    """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
    return app_home().joinpath(*[str(p) for p in parts])


def default_config_path() -> Path:
    """``<app-home>/config.json`` ? the config file the loader reads and ``--setup`` writes.

    With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative), unchanged.
    """
    return app_home() / "config.json"


def default_env_path() -> Path:
    """``<app-home>/.env`` ? the dotenv file the server prefers (e.g. ``GEMINI_API_KEY``)."""
    return app_home() / ".env"


def env_file_to_load() -> Path | None:
    """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's DEFAULT
    (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()`` behaviour.

    Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file exists. With
the
    env UNSET this returns ``None`` so the caller keeps the historical search and does NOT newly
    prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file, not the
CWD).
    """
    home = app_home()
    if (
        str(home) == "."
    ):  # env unset (or pinned to CWD) ? historical load_dotenv() search
        return None
    candidate = home / ".env"
    return candidate if candidate.exists() else None


def resolve_output_dir(output_dir: str) -> str:
    """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one through.

    With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default ``"output"`` when
    empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no pathlib
    normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir like
    ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the rest of
the
    app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` / ``C:x``) ?
```

passes

7. user (2026-07-02T08:18:34.015Z)

```
=== grep resolve_output_dir in uploads.py ===
25:    resolve_output_dir,  # P0a: app-home-aware path resolution
53:    upload_dir = resolve_output_dir(
=== models.py 328-340 ===
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
=== tests grep ===
```

8. assistant (2026-07-02T08:18:46.246Z)

Verification complete. The evidence strongly confirms the analyst.

Functional core on master (already present):
- `backend/api/uploads.py:24-26` imports `resolve_output_dir` from `backend.utils.app_home` (multi-line form, but identical import).
- `backend/api/uploads.py:53-55` wraps the exact expression: `upload_dir = resolve_output_dir(getattr(sec, "upload_dir", None) or "output/uploads")`.
- Introduced by merged PR #422 (commit 4f148d1) ? `git log -S resolve_output_dir` returns only that commit.
- `resolve_output_dir` semantics in `app_home.py:83+` confirm P0a: RALLY_APP_HOME unset ? verbatim; relative ? under home; absolute ? passthrough. Matches the PR's stated intent exactly.

The PR #415 uploads.py hunk is functionally byte-equivalent to what master already has (only cosmetic: single-line import + longer inline comment). It conflicts precisely because master already applied the same fix on the identical lines ? classic two-sessions-same-fix collision.

Residuals NOT on master (all non-functional):
- `models.py:333-335` ? master still has the OLD comment (no RALLY_APP_HOME / resolve_output_dir mention). Confirmed.
- `docs/CODE_MAP.md` ? master lacks the appended P0a sentence.
- 4 new tests in `tests/test_upload_download.py` ? grep for `test_upload_cfg_relative_default_roots_under_app_home` / `test_upload_lands_under_app_home_not_launch_cwd` / `APP_HOME_ENV` returns nothing on master.

The load-bearing behavior is fully on master. The only losses from closing are doc-comment polish plus 4 tests that verify behavior master already exhibits (not a stricter guard, not a different code path, no functional gap). This does not justify rebasing the whole PR. Analyst's CLOSE_SUPERSEDED (superseded_by=422) is correct. I would note the additive test/doc hunks are worth cherry-picking as a tiny follow-up, but that doesn't change the fate.

9. user (2026-07-02T08:18:53.055Z)

Structured output provided successfully